Technical Design Note

AI-Powered Document Intelligence Assistant

Sensiwise AI – Junior Full-Stack Developer Assessment

1. Design Decisions

The application was designed around three core priorities: correctness of the RAG pipeline, developer ergonomics, and a clean user experience. The system is split into two clearly separated layers — a Next.js frontend and a Bun + Express backend — communicating over a REST API. This separation keeps concerns clean and allows each layer to be developed, tested, and scaled independently.

The RAG pipeline follows a standard ingest-then-query pattern. On upload, documents are parsed, chunked, embedded, and stored in Qdrant. At query time, the user's question is embedded, the nearest chunks are retrieved by cosine similarity, and those chunks are injected into the prompt sent to the LLM. The answer includes inline references to the source chunks, giving the user traceability back to the original document.

Streaming was implemented end-to-end: the OpenAI response streams token-by-token from the backend to the frontend over Server-Sent Events (SSE). This makes the interface feel responsive and avoids long blank waits on lengthy answers.

2. Technology Choices

Frontend — Next.js

Next.js was chosen for its strong developer experience, built-in routing, and SSE-friendly API routes. The chat UI is a single-page interface with a file panel on the left and a conversation area on the right, built with Tailwind CSS for rapid, consistent styling.

Backend — Bun + Express

Bun was selected as the runtime for its significantly faster startup and install times compared to Node.js, while remaining fully compatible with the Express ecosystem. Express provides familiar, lightweight HTTP routing without the overhead of a full framework.

Vector Store — Qdrant

Qdrant was chosen over alternatives such as Chroma or FAISS because it ships as a standalone Docker container with a clean REST and gRPC API, supports payload filtering, and is straightforward to self-host. It handles cosine similarity search efficiently even at the scale of a prototype.

AI Layer — OpenAI

OpenAI's text-embedding-3-small model is used for generating chunk and query embeddings, and gpt-4o-mini is used for answer generation. This combination offers strong quality at low cost and latency. The streaming API is used for all completions.

Containerisation — Docker

A docker-compose.yml file orchestrates both the backend and Qdrant, making local setup a single command. The frontend is run separately with bun dev or can be added as a third service. This ensures the application is portable and reproducible across environments.

3. Challenges Encountered

The most significant challenge was chunking PDF content accurately. PDFs do not have a clean semantic structure — text is stored as positioned glyphs rather than paragraphs or sentences. Libraries such as pdf-parse produce raw text that loses paragraph boundaries, merges columns, and sometimes reorders content. Two specific problems arose:

- Headers, footers, and page numbers were extracted alongside body text, polluting chunks with irrelevant content.

- Fixed-size character chunking cut across sentences mid-word, producing fragments that degraded retrieval quality.

This was partially addressed by applying a sentence-boundary-aware chunking strategy with overlapping windows (chunk size ~500 tokens, overlap ~50 tokens). This reduced mid-sentence cuts and improved context continuity within each chunk. However, it did not fully resolve layout noise from complex PDFs.

4. What I Would Improve With More Time

- Better chunking strategy: Replace the current text-based chunking with a layout-aware PDF parser (such as PDFMiner or a vision-based approach) to accurately extract structure, ignore headers/footers, and preserve paragraph boundaries.
- Conversation memory: Maintain a per-session chat history and inject a summarised context window into each prompt, enabling follow-up questions that refer back to earlier answers.
- Multi-document support: Allow the user to upload and query across several documents simultaneously, with per-document filtering at the Qdrant payload level.
- Authentication and sessions: Add user accounts so each user's documents and conversation history are isolated and persisted across sessions.
- Cloud deployment: Deploy the backend and Qdrant to a managed environment (such as Railway or Render) and the frontend to Vercel, with environment secrets managed via a secrets manager rather than a .env file.